

TaxiTap System
Service Contracts Documentation
Software Requirements Specification



Contents

1	Introduction	4
2	User Account Management Service	4
2.1	signUpSMS Function	4
2.2	loginSMS Function	4
2.3	switchActiveRole Function	5
2.4	updateUserProfile Function	5
3	Ride Request System Service	6
3.1	requestRideHandler Function	6
3.2	acceptRideHandler Function	7
3.3	cancelRideHandler Function	8
3.4	completeRideHandler Function	9
3.5	declineRideHandler Function	10
3.6	endRideHandler Function	10
4	Notification System Service	11
4.1	sendNotificationHandler Function	11
4.2	getNotificationsHandler Function	12
4.3	markNotificationAsReadHandler Function	13
5	Routes Service	14
5.1	calculateRoute Function	14
5.2	displayRoute Function	14
5.3	storeRecentRoute Function	15
6	Payment System Service	16
6.1	earnings Function	16
6.2	fare Function	17
7	Rating and Feedback System Service	17
7.1	averageRating Function	17
7.2	saveFeedback Function	18
7.3	showFeedback Function	19
8	Badge Service	20
8.1	getUserBadges (Query)	20
8.2	getBadgeInfo (Query)	20
8.3	awardBadge (Mutation)	20
8.4	checkAndAwardTrustedPayerBadge (Mutation)	21
8.5	checkAndAwardMarathonDriverBadge (Mutation)	21
8.6	updateTopEarnerBadges (Mutation - Internal/Scheduled)	21
9	Journeys Service	22
9.1	findMultiLegJourneyOptionsHandler Query	22
9.2	createMultiLegJourney Mutation	23
9.3	getNextLegInfo Query	23
9.4	startJourneyLeg Mutation	23
9.5	completeLegWithPayment Mutation	24
9.6	cancelJourney Mutation	24
9.7	handleTransferTimeout Mutation	25

9.8	getJourneyState Query	25
9.9	getActiveJourneyForPassenger Query	25
9.10	cleanupExpiredTransfers Mutation (Internal)	26
10	Conclusion	26

1 Introduction

This document provides service contracts for all major services within the TaxiTap ride-sharing system. Each service contract defines a clear and consistent description of how the service can be used, including its purpose, required inputs, expected outputs, and interactions with other system components.

Service contracts serve as API specifications that outline the interface and behavior expectations for each service, ensuring clear communication between different parts of the system and external integrators.

2 User Account Management Service

The User Account Management service handles all user-related operations including registration, authentication, profile management, and role switching functionality.

2.1 signUpSMS Function

Field	Description
What the service does	Registers a new user account in the TaxiTap system using SMS-based verification. Creates user profile with provided information and sends SMS verification code to the provided phone number.
Inputs required	• phoneNumber (string) - Valid phone number in international format • name (string) - User's full name (2-0 characters) • password (string) - Secure password (minimum 8 characters) • accountType ("passenger" — "driver" — "both") - Account role type • email (optional string) - Valid email address • age (optional number) - User age (must be 18 or older)
Outputs returned	• userId (Id;"taxiTap_users";) - Unique user identifier • verificationStatus (boolean) - SMS verification status • sessionToken (string) - Authentication token for session • userProfile (object) - Created user profile data • success (boolean) - Operation success indicator • errorMessage (optional string) - Error details if operation fails
Interactions with other systems/components	• Database Service - Stores user account data • Authentication Service - Authenticates user information • Validation Service - Validates input parameters • Notification System - Sends welcome notification

2.2 loginSMS Function

Field	Description
-------	-------------

What the service does	Authenticates existing users using phone number and password combination. Validates credentials and establishes authenticated session with appropriate permissions based on user's account type.
Inputs required	• phoneNumber (string) - Registered phone number • password (string) - User password
Outputs returned	• userId (Id "taxiTap_users") - Authenticated user identifier • sessionToken (string) - Authentication token for session • userProfile (object) - User profile information • activeRole ("passenger" — "driver") - Current active role • permissions (array) - User permission set • success (boolean) - Authentication result • errorMessage (optional string) - Error details if authentication fails
Interactions with other systems/components	• Database Service - Retrieves and validates user credentials • Authentication Service - Generates and manages session tokens • Security Service - Logs authentication attempts • User Profile Service - Retrieves user profile data

2.3 switchActiveRole Function

Field	Description
What the service does	Allows users with multiple account types to switch between passenger and driver roles. Updates active permissions and interface preferences based on selected role.
Inputs required	• userId (Id "taxiTap_users") - Authenticated user identifier • newRole ("passenger" — "driver") - Target role to switch to
Outputs returned	• activeRole ("passenger" — "driver") - Updated active role • permissions (array) - Updated permission set • interfaceConfig (object) - Role-specific UI configuration • success (boolean) - Role switch result • errorMessage (optional string) - Error details if switch fails
Interactions with other systems/components	• Authentication Service - Updates session permissions • Database Service - Updates user's active role status • UI Configuration Service - Provides role-specific interface settings • Activity Tracking Service - Logs role switch event

2.4 updateUserProfile Function

Field	Description
-------	-------------

What the service does	Updates user profile information including personal details, contact information, and emergency contacts. Validates all changes and maintains data integrity across the system.
Inputs required	• userId (Id "taxiTap_users") - User identifier for profile update • name (string) - Updated full name • phoneNumber (string) - Updated phone number • email (optional string) - Updated email address • profilePicture (optional string) - Profile image URL or base64 data • emergencyContact (optional object) - Emergency contact details: – name (string) - Contact person's name – phoneNumber (string) - Contact phone number – relationship (string) - Relationship to user
Outputs returned	• updatedProfile (object) - Complete updated user profile • changedFields (array) - List of fields that were modified • validationErrors (array) - Any validation issues encountered • success (boolean) - Update operation result • errorMessage (optional string) - Error details if update fails
Interactions with other systems/components	• Database Service - Persists profile changes • Validation Service - Validates all input data • Image Processing Service - Processes profile picture uploads • Notification System - Notifies user of successful profile updates • Security Service - Logs profile modification events

3 Ride Request System Service

The Ride Request System service manages the complete ride request lifecycle from initial passenger request through driver matching, ride execution, and completion. This service consists of multiple handler functions that coordinate real-time interactions between passengers and drivers.

3.1 requestRideHandler Function

Field	Description
What the service does	Initiates a new ride request from a passenger. Creates a ride record, calculates fare estimates, searches for available drivers in the vicinity, and broadcasts the ride request to eligible drivers. Manages the initial matching process and sets up real-time tracking.

Inputs required	• passengerId (string) - Unique identifier of the requesting passenger • driverId (string) - Specific driver identifier if passenger has a preference • startLocation (object) - Pickup location details: – coordinates (object) - Latitude and longitude coordinates – address (string) - Human-readable pickup address • endLocation (object) - Destination location details: – coordinates (object) - Latitude and longitude coordinates – address (string) - Human-readable destination address • estimatedFare (optional number) - Pre-calculated fare estimate • estimatedDistance (optional number) - Pre-calculated distance in kilometers • estimatedDuration (optional number) - Pre-calculated duration in minutes
Outputs returned	• rideId (string) - Unique identifier for the created ride request • requestStatus ("created" — "searching" — "pending_driver_response") - Current request status • estimatedMatchTime (number) - Expected time to find a driver in minutes • nearbyDrivers (array) - List of available drivers in the area • fareDetails (object) - Detailed fare breakdown including base fare, taxes, and fees • routePreview (object) - Basic route information and map data • success (boolean) - Request creation result • errorMessage (optional string) - Error details if request fails
Interactions with other systems/components	• Database Service - Creates ride record and retrieves passenger details • Driver Matching Service - Finds and filters available drivers • Routes Service - Calculates route and fare estimates • Notification System - Sends ride requests to drivers • Real-time Tracking Service - Initializes location tracking • Geolocation Service - Validates and processes location data • Fare Calculation Service - Computes accurate pricing

3.2 acceptRideHandler Function

Field	Description
What the service does	Processes a driver's acceptance of a ride request. Updates ride status, establishes the passenger-driver connection, cancels notifications to other drivers, and initiates the active ride phase with real-time coordination.

Inputs required	• rideId (string) - Unique identifier of the ride request being accepted • driverId (string) - Unique identifier of the driver accepting the ride
Outputs returned	• rideStatus ("accepted" — "driver_en_route") - Updated ride status • driverDetails (object) - Complete driver profile and vehicle information • estimatedArrival (datetime) - Expected driver arrival time at pickup location • trackingInfo (object) - Real-time tracking details for passenger • contactInfo (object) - Secure communication channels between passenger and driver • ridePin (string) - Security PIN for ride verification • success (boolean) - Acceptance processing result • errorMessage (optional string) - Error details if acceptance fails
Interactions with other systems/components	• Database Service - Updates ride status and driver assignment • Notification System - Notifies passenger of driver acceptance and cancels other driver notifications • Real-time Tracking Service - Activates live location sharing • Driver Availability Service - Updates driver status to busy • Communication Service - Establishes secure passenger-driver communication • Route Optimization Service - Provides optimal route to pickup location

3.3 cancelRideHandler Function

Field	Description
What the service does	Handles ride cancellation requests from either passengers or drivers. Processes cancellation fees if applicable, updates all parties involved, releases driver availability, and manages refunds or charges based on cancellation policy and timing.
Inputs required	• rideId (string) - Unique identifier of the ride to be cancelled • userId (string) - Identifier of the user initiating the cancellation (passenger or driver)

Outputs returned	• cancellationStatus ("cancelled" — "cancellation_failed") - Result of cancellation attempt • affectedParties (array) - List of users notified about the cancellation • driverCompensation (optional object) - Compensation for driver if applicable • success (boolean) - Cancellation processing result • errorMessage (optional string) - Error details if cancellation fails
Interactions with other systems/components	• Database Service - Updates ride status and records cancellation details • Payment System - Processes cancellation fees and refunds • Notification System - Notifies all affected parties of cancellation • Seat Availability Service - Updates driver seat status back to available • Real-time Tracking Service - Terminates active tracking sessions

3.4 completeRideHandler Function

Field	Description
What the service does	Finalizes a ride when the driver marks it as completed after reaching the destination. Calculates final fare based on actual route and time, processes payment, updates driver availability, and triggers post-ride activities like rating requests.
Inputs required	• rideId (string) - Unique identifier of the ride being completed • driverId (string) - Identifier of the driver marking the ride as complete
Outputs returned	• rideStatus ("completed") - Final ride status • finalFare (object) - Complete fare breakdown including actual charges • paymentStatus ("processed" — "pending" — "failed") - Payment processing result • rideHistory (object) - Complete ride details for records • receiptInfo (object) - Digital receipt details • driverEarnings (object) - Driver earnings breakdown • ratingPrompt (boolean) - Whether rating requests should be sent • success (boolean) - Completion processing result

Interactions with other systems/components	• Database Service - Updates ride completion status and stores final details • Payment System - Processes final payment and driver earnings • Notification System - Sends completion notifications and rating requests • Real-time Tracking Service - Finalizes tracking data
---	--

3.5 declineRideHandler Function

Field	Description
What the service does	Processes a driver's decision to decline a ride request. Records the decline reason, removes the driver from the current request pool, continues searching for alternative drivers, and may adjust driver matching algorithms based on decline patterns.
Inputs required	• rideId (string) - Unique identifier of the ride request being declined • driverId (string) - Identifier of the driver declining the ride
Outputs returned	• declineStatus ("declined") - Confirmation of decline processing • remainingDrivers (array) - List of other drivers still available for the ride • searchContinues (boolean) - Whether the search for drivers continues • estimatedMatchTime (number) - Updated estimated time to find a driver • driverAvailability ("available" — "temporarily_unavailable") - Driver's updated availability status • success (boolean) - Decline processing result • errorMessage (optional string) - Error details if decline processing fails
Interactions with other systems/components	• Database Service - Records decline and updates ride request status • Driver Matching Service - Continues search with remaining drivers • Analytics Service - Records decline patterns for algorithm improvement • Notification System - May notify passenger of continued search

3.6 endRideHandler Function

Field	Description
What the service does	Handles the final termination of a ride session, typically called after completion processing. Cleans up active sessions, terminates real-time connections, archives ride data, and ensures all resources are properly released for system efficiency.

Inputs required	• rideId (string) - Unique identifier of the ride session to end • userId (string) - Identifier of the user requesting to end the ride session
Outputs returned	• sessionStatus ("ended") - Confirmation that ride session has been terminated • dataArchived (boolean) - Whether ride data has been successfully archived • resourcesReleased (boolean) - Confirmation of system resource cleanup • finalSummary (object) - Complete ride summary for user records • nextActions (array) - Available post-ride actions (rate, report, share) • success (boolean) - Session termination result • errorMessage (optional string) - Error details if termination fails
Interactions with other systems/components	• Database Service - Archives ride data and cleans active sessions • Real-time Tracking Service - Terminates location tracking and connections • Session Management Service - Ends active ride sessions • Resource Management Service - Releases allocated system resources • Cache Service - Clears temporary ride-related cached data • Monitoring Service - Updates system performance metrics • Cleanup Service - Performs post-ride system maintenance tasks

4 Notification System Service

The Notification System service manages all communication between the system and users through multiple channels including SMS, push notifications, and in-app messages. This service handles notification delivery, user preferences, and message tracking.

4.1 sendNotificationHandler Function

Field	Description
What the service does	Sends notifications to users through appropriate channels based on user preferences and message type. Handles message formatting, delivery method selection, queuing for reliable delivery, and tracks delivery status across multiple notification channels.
Inputs required	• userId (string) - Unique identifier of the target user • message (string) - The notification message content to be sent • type (string) - Notification category such as "ride", "payment", "system" • metadata (optional object) - Additional data for message customization and tracking

Outputs returned	• notificationId (string) - Unique identifier for the sent notification • deliveryStatus ("queued" — "sent" — "delivered" — "failed") - Current delivery status • deliveryMethod (string) - Method used for delivery (SMS, push, in-app, email) • deliveryTimestamp (datetime) - When the notification was sent • estimatedDelivery (datetime) - Expected delivery time • retryCount (number) - Number of delivery attempts if failed • success (boolean) - Notification sending result • errorMessage (optional string) - Error details if sending fails
Interactions with other systems/components	• Database Service - Stores notification records and retrieves user preferences • SMS Gateway - Sends SMS notifications • Push Notification Service - Delivers mobile push notifications • Email Service - Sends email notifications • User Preferences Service - Determines preferred notification channels • Template Engine - Formats messages using predefined templates • Queue Management Service - Manages notification delivery queues • Analytics Service - Tracks notification metrics and engagement

4.2 getNotificationsHandler Function

Field	Description
What the service does	Retrieves a user's notification history including unread messages, notification status, and metadata. Supports pagination for large notification lists and provides filtering options by notification type, date range, and read status.
Inputs required	• userId (string) - Unique identifier of the user whose notifications to retrieve
Outputs returned	• notifications (array) - List of user notifications with details • unreadCount (number) - Total number of unread notifications • totalCount (number) - Total number of notifications for the user • latestNotification (object) - Most recent notification details • notificationTypes (array) - Available notification categories • paginationInfo (object) - Pagination details for large lists • success (boolean) - Retrieval operation result • errorMessage (optional string) - Error details if retrieval fails

Interactions with other systems/components	• Database Service - Retrieves stored notification records • User Authentication Service - Validates user identity and permissions • Pagination Service - Handles large result set pagination • Notification Filtering Service - Applies filters and sorting options • Cache Service - Caches frequently accessed notification data • Analytics Service - Tracks notification access patterns
---	--

4.3 markNotificationAsReadHandler Function

Field	Description
What the service does	Updates the read status of a specific notification when a user views it. Records the read timestamp, updates unread counts, and may trigger follow-up actions based on notification type. Ensures accurate tracking of user engagement with notifications.
Inputs required	• notificationId (string) - Unique identifier of the notification to mark as read • userId (string) - Identifier of the user marking the notification as read
Outputs returned	• readStatus ("read") - Confirmation that notification has been marked as read • readTimestamp (datetime) - When the notification was marked as read • updatedUnreadCount (number) - New unread notification count for the user • followUpActions (array) - Any triggered actions based on the notification type • notificationDetails (object) - Updated notification information • success (boolean) - Read status update result • errorMessage (optional string) - Error details if update fails
Interactions with other systems/components	• Database Service - Updates notification read status and timestamp • User Authentication Service - Validates user permissions for the notification • Analytics Service - Tracks notification engagement and read patterns • Counter Service - Updates unread notification counters • Follow-up Action Service - Triggers actions based on notification type • Cache Service - Updates cached notification status data

5 Routes Service

5.1 calculateRoute Function

Field	Description
Service Name	findBestMatchingRoutes
What the service does	Finds optimal transportation routes based on geographical proximity to user-specified start and end locations. Calculates distance-based scores using Haversine formula, filters routes within configurable distance limits, and returns routes sorted by suitability with comprehensive proximity metrics and stop details.
Inputs required	• startLat (number) - Latitude of departure location • startLon (number) - Longitude of departure location • endLat (number) - Latitude of destination location • endLon (number) - Longitude of destination location • maxStartDistance (optional number) - Maximum acceptable distance from start point in km (default: 2.0) • maxEndDistance (optional number) - Maximum acceptable distance from end point in km (default: 8.0) • maxResults (optional number) - Maximum routes to return (default: 10)
Outputs returned	• success (boolean) - Operation status • matchingRoutes (array) - Optimized route recommendations with: – Route identification and metadata – Proximity measurements to start/end points – Closest stop details with distances – Route suitability scoring – Direct route availability flag • totalRoutesChecked (number) - Total active routes evaluated • routesWithinRange (number) - Routes meeting distance criteria • searchCriteria (object) - Parameters used for search • message (string) - Status or error information
Interactions with other systems/components	• Route Database - Retrieves all active routes • Enriched Stops Database - Gets enhanced stop information • Distance Calculation Service - Haversine formula computations • Scoring Engine - Calculates route suitability scores • Filtering Service - Applies distance constraints • Sorting Service - Orders results by priority

5.2 displayRoute Function

Field	Description
-------	-------------

What the service does	Provides comprehensive details for a specific route including all stops with optional distance calculations from a user's current location. Retrieves enriched stop information and calculates proximity metrics when user coordinates are provided.
Inputs required	• routeId (string) - Unique identifier of the target route • userLat (optional number) - User's current latitude for distance calculations • userLon (optional number) - User's current longitude for distance calculations
Outputs returned	• success (boolean) - Operation status • route (object) - Complete route details including: – Route metadata and operational information – Complete stop list with ordering – Distance calculations from user (if coordinates provided) – Enrichment status indicator • message (string) - Status or error information
Interactions with other systems/components	• Route Database - Retrieves specific route by ID • Enriched Stops Database - Gets enhanced stop data • Distance Calculation Service - Computes user-to-stop distances • Data Enrichment Service - Provides stop name enhancement status • Error Handling - Manages missing route scenarios

5.3 storeRecentRoute Function

Field	Description
What the service does	Discovers all transportation stops within a specified radius of a given location across all active routes. Provides comprehensive stop information including associated route details and exact distance measurements.
Inputs required	• lat (number) - Center point latitude for search • lon (number) - Center point longitude for search • radiusKm (optional number) - Search radius in kilometers (default: 2.0) • maxResults (optional number) - Maximum stops to return (default: 20)

Outputs returned	• success (boolean) - Operation status • nearbyStops (array) - Stop objects within radius containing: – Stop identification and metadata – Precise distance measurement – Associated route information – Geographical coordinates • searchLocation (object) - Center coordinates used for search • radiusKm (number) - Search radius applied • totalFound (number) - Number of stops discovered • message (string) - Status or error information
Interactions with other systems/components	• Route Database - Retrieves all active routes • Enriched Stops Database - Gets enhanced stop data • Geographical Search Service - Radial distance calculations • Result Aggregation - Collects stops from multiple routes • Sorting Service - Orders stops by proximity • Pagination Service - Limits results according to parameters

6 Payment System Service

6.1 earnings Function

Field	Description
Service Name	getWeeklyEarnings
What the service does	Calculates comprehensive weekly earnings and performance metrics for a driver over the last 4 weeks. Aggregates trip data, work sessions, and provides detailed daily breakdowns including earnings, online hours, reservations, and hourly averages.
Inputs required	• driverId (string) - Unique identifier of the driver
Outputs returned	• Array of weekly objects containing: – dateRangeStart (number) - Start timestamp of the week – earnings (number) - Total earnings for the week – hoursOnline (number) - Total hours worked (rounded) – averagePerHour (number) - Average earnings per hour – reservations (number) - Number of reservation trips – dailyData (array) - Daily breakdown with earnings per day – todayEarnings (number) - Earnings for current day (current week only)

Interactions with other systems/components	• Trip Database - Retrieves driver's trip records • Work Sessions Database - Gets driver's work session data • Date/Time Service - Calculates week boundaries and day ranges • Aggregation Service - Computes totals and averages • Data Transformation - Formats daily breakdowns
Special Features	• Handles 4-week rolling window analysis • Calculates real-time today's earnings for current week • Provides day-by-day earnings breakdown • Computes hourly productivity metrics • Differentiates reservation vs regular trips

6.2 fare Function

Field	Description
Service Name	getFareForLatestTripHandler
What the service does	Retrieves the fare amount from the most recent trip for a given user, checking both passenger and driver roles. Returns the latest trip fare regardless of whether the user was a passenger or driver in that trip.
Inputs required	• userId (string) - Unique identifier of the user
Outputs returned	• fare (number) - Fare amount from the most recent trip • null - If no trips found for the user
Interactions with other systems/components	• Trip Database - Queries trip records by passengerId • Trip Database - Queries trip records by driverId • User Role Service - Checks user's potential roles in trips • Sorting Service - Orders trips by recency (descending) • Result Resolution - Returns first non-null result from role checks
Special Features	• Dual-role support (passenger and driver) • Returns most recent trip regardless of role • Graceful null handling for users with no trip history • Efficient querying with database indexes

7 Rating and Feedback System Service

7.1 averageRating Function

Field	Description
-------	-------------

What the service does	Calculates the average rating for a specific driver based on all submitted feedback. Processes all valid ratings, computes the arithmetic mean, and returns the result formatted to one decimal place. Returns 0 if no ratings are available.
Inputs required	• <code>driverId</code> (Id) - Unique identifier of the driver
Outputs returned	• <code>averageRating</code> (number) - Calculated average rating (0.0 to .0) • Returns 0 if no valid ratings found
Interactions with other systems/components	• Feedback Database - Retrieves all feedback records for the driver • Data Validation Service - Filters out invalid or non-numeric ratings • Calculation Service - Computes arithmetic mean • Formatting Service - Rounds result to one decimal place
Special Features	• Handles empty rating sets gracefully • Filters invalid ratings (non-numbers, zero/negative values) • Precision formatting to one decimal place • Efficient database indexing by driver
Error Handling	• Returns 0 instead of throwing errors for no ratings • Silent filtering of invalid rating values

7.2 saveFeedback Function

Field	Description
What the service does	Stores passenger feedback for a completed ride. Ensures only one feedback submission per ride, validates input data, and creates a comprehensive feedback record with rating, comments, and location details.
Inputs required	• <code>rideId</code> (string) - Unique identifier of the ride • <code>passengerId</code> (string) - Identifier of the passenger submitting feedback • <code>driverId</code> (string) - Identifier of the driver being rated • <code>rating</code> (number) - Numerical rating (typically 1-) • <code>comment</code> (string) - Optional textual feedback • <code>startLocation</code> (object) - Ride starting location details • <code>endLocation</code> (object) - Ride ending location details
Outputs returned	• <code>id</code> (string) - Unique identifier of the created feedback record • Throws error if feedback already exists for the ride

Interactions with other systems/components	• Feedback Database - Checks for existing feedback by rideId • Database Insertion Service - Creates new feedback record • Validation Service - Ensures ride-based uniqueness • Timestamp Service - Records creation time
Special Features	• Prevents duplicate feedback for the same ride • Comprehensive ride context preservation • Automatic timestamping • Ride-based indexing for efficient lookups
Error Conditions	• "Feedback already submitted for this ride" - Duplicate prevention • Database insertion failures • Invalid input data types

7.3 showFeedback Function

Field	Description
What the service does	Provides two specialized functions for retrieving feedback: one for passengers to view their submitted feedback with driver details, and one for drivers to view feedback received about their service.
Components	• <code>showFeedbackPassengerHandler</code> - Gets feedback submitted by a passenger • <code>showFeedbackDriverHandler</code> - Gets feedback received by a driver
Inputs required	• <code>passengerId</code> (string) - For passenger view • <code>driverId</code> (string) - For driver view
Outputs returned	• Array of feedback objects containing: – Rating, comments, timestamps – Ride location details – Driver information (passenger view only) – Chronological ordering (descending)
Interactions with other systems/components	• Feedback Database - Retrieves feedback records by user • User Database - Enriches feedback with driver details (passenger view) • Data Enrichment Service - Adds driver names to passenger feedback
Special Features	• Dual perspective: passenger and driver views • Chronological sorting for easy review • Efficient database indexing by user IDs

8 Badge Service

The Badge Service is responsible for managing, tracking, and awarding digital badges to users (both drivers and passengers) based on achievements, performance, and loyalty within the TaxiTap ecosystem. Badges are derived from specific criteria evaluated by helper functions.

8.1 getUserBadges (Query)

Field	Description
What the service does	Retrieves all currently active and earned badges for a specific user, enriching the badge record with static metadata from <code>BADGE_DEFINITIONS</code> .
Inputs required	• <code>userId</code> (Id: "taxiTap-users") - The ID of the user whose badges are requested.
Outputs returned	• Array of Badge objects containing: <code>badgeType</code>, <code>name</code>, <code>description</code>, <code>icon</code>, <code>color</code>, <code>earnedAt</code> (timestamp), <code>isActive</code>, and <code>metadata</code> (optional).
Interactions with other systems/components	• Database Service - Queries the <code>badges</code> table by user ID.

8.2 getBadgeInfo (Query)

Field	Description
What the service does	Retrieves the static definition and display information for a specific badge type (e.g., name, icon, color).
Inputs required	• <code>badgeType</code> (string) - The unique identifier of the badge definition.
Outputs returned	• <code>BadgeInfo</code> (object or null) - Static badge definition details.
Interactions with other systems/components	• None - Uses local application constants (<code>BADGE_DEFINITIONS</code>).

8.3 awardBadge (Mutation)

Field	Description
What the service does	The core mutation to award a badge. It creates a new badge record or reactivates an existing badge of the same type for the user, updating the <code>earnedAt</code> timestamp.

Inputs required	• userId (Id!"taxiTap_users"!) - The ID of the user to receive the badge. • badgeType (string) - The unique identifier of the badge to be awarded. • metadata (optional any) - Contextual data related to the badge award (e.g., total rides).
Outputs returned	• void
Interactions with other systems/components	• Database Service - Queries and patches/inserts records in the badges table.

8.4 checkAndAwardTrustedPayerBadge (Mutation)

Field	Description
What the service does	Automated check to determine if a passenger is eligible for the trusted_payer badge (100% payment rate on completed rides). Awards the badge via awardBadge if eligible.
Inputs required	• userId (Id!"taxiTap_users"!) - The passenger's ID.
Outputs returned	• boolean - True if the badge was awarded, false otherwise.
Interactions with other systems/components	• Database Service - Queries the rides table and updates the badges table.

8.5 checkAndAwardMarathonDriverBadge (Mutation)

Field	Description
What the service does	Automated check to determine if a driver is eligible for the marathon_driver badge (completed at least one ride). Awards the badge via awardBadge if eligible.
Inputs required	• userId (Id!"taxiTap_users"!) - The driver's ID.
Outputs returned	• boolean - True if the badge was awarded, false otherwise.
Interactions with other systems/components	• Database Service - Queries the rides table and updates the badges table.

8.6 updateTopEarnerBadges (Mutation - Internal/Scheduled)

Field	Description
What the service does	A background process that periodically recalculates the top 10 earning drivers. It activates the <code>top_earner</code> badge for qualifying drivers and deactivates it for those who are no longer in the top 10.
Inputs required	• None.
Outputs returned	• <code>void</code>
Interactions with other systems/components	• Database Service - Queries <code>taxiTap_users</code>, <code>trips</code>, and patches/inserts the <code>badges</code> table for all drivers.

9 Journeys Service

The Journeys Service manages the complex lifecycle of multi-leg taxi journeys, including planning, state tracking, and leg-by-leg progression. It is crucial for cross-regional travel requiring transfers between different route associations.

9.1 findMultiLegJourneyOptionsHandler Query

Field	Description
What the service does	Calculates potential multi-leg journey routes between an origin and a destination that cannot be serviced by a single taxi route, involving a transfer point and walking legs.
Inputs required	• <code>originLat</code>, <code>originLng</code> (number) - Start coordinates • <code>destinationLat</code>, <code>destinationLng</code> (number) - End coordinates • <code>maxWalkingDistance</code> (optional number) - Max walking distance from origin/to destination (default 1km) • <code>maxTransferDistance</code> (optional number) - Max walking distance between transfer stops (default 4km)
Outputs returned	• <code>success</code> (boolean) - Indicates if options were successfully found • <code>journeyOptions</code> (array) - List of viable <code>MultiLegOption</code> objects, including route names, stop IDs, and estimated costs • <code>message</code> (optional string) - Explanatory message for success/failure • <code>searchCriteria</code> (object) - Echoes the input criteria used for the search
Interactions with other systems/components	• Database Service - Queries <code>routes</code>, <code>enrichedRouteStops</code>, <code>locations</code>, and <code>drivers</code> tables • Geospatial Service - Performs complex distance and fare calculations

9.2 createMultiLegJourney Mutation

Field	Description
What the service does	Initializes and persists a new multi-leg journey record based on a planned travel option, detailing both legs, the transfer point, and estimated total cost.
Inputs required	• passengerId (Id!“taxiTap_users”!) - The ID of the passenger initiating the journey • journeyOption (object) - A structure defining the two legs, their stops, estimated costs, and transfer point details
Outputs returned	• journeyId (string) - The user-facing unique journey identifier • dbId (Id!“multiLegJourneys”!) - The internal database ID of the new journey document
Interactions with other systems/components	• Database Service - Inserts a new record into the multiLegJourneys table

9.3 getNextLegInfo Query

Field	Description
What the service does	Retrieves detailed information about the subsequent leg of a multi-leg journey, including route details and available drivers on that route, specifically for display on the TaxiInformation screen.
Inputs required	• journeyId (string) - Unique identifier for the multi-leg journey • currentLegIndex (number) - The 0-based index of the currently completed or active leg
Outputs returned	• hasNextLeg (boolean) - True if a subsequent leg exists • message (optional string) - Status message (e.g., “This is the final leg...”) • nextLeg (optional object) - Details of the next leg (route, origin, destination, cost, etc.) • availableDrivers (array) - List of drivers available on the next leg’s route • journey (optional object) - Current state summary of the journey
Interactions with other systems/components	• Database Service - Retrieves journey, route, location, driver, and taxi data • Location/Geospatial Service - Implicitly uses location data for driver matching

9.4 startJourneyLeg Mutation

Field	Description
What the service does	Transitions a specific leg of the journey from 'pending' to 'in_progress', associating it with a specific ride and driver. Updates the overall journey status and current leg index.
Inputs required	• <code>journeyId</code> (string) - Unique identifier for the multi-leg journey • <code>legIndex</code> (number) - The 0-based index of the leg being started • <code>rideId</code> (Id "rides" i) - The ID of the corresponding single-leg ride record • <code>driverId</code> (Id "taxiTap-users" i) - The ID of the driver assigned to this leg
Outputs returned	• <code>success</code> (boolean) - True if the leg was successfully started
Interactions with other systems/components	• Database Service - Patches the <code>multiLegJourneys</code> document with updated status and leg details

9.5 completeLegWithPayment Mutation

Field	Description
What the service does	Marks a specific journey leg as 'completed', records the final actual cost, and updates the total journey cost. If it's the final leg, the entire journey status is set to 'completed'.
Inputs required	• <code>journeyId</code> (string) - Unique identifier for the multi-leg journey • <code>legIndex</code> (number) - The 0-based index of the leg being completed • <code>actualCost</code> (number) - The final cost of this completed leg
Outputs returned	• <code>success</code> (boolean) - True if the leg was successfully completed • <code>journeyComplete</code> (boolean) - True if this was the final leg and the journey is finished • <code>nextLeg</code> (optional object) - Details about the next leg if the journey continues • <code>totalActualCost</code> (optional number) - The cumulative cost of all completed legs
Interactions with other systems/components	• Database Service - Patches the <code>multiLegJourneys</code> document with updated cost and status • Payment Service - Assumed to integrate with this step to finalize payment

9.6 cancelJourney Mutation

Field	Description
-------	-------------

What the service does	Updates the overall journey status to 'cancelled' and marks any 'pending' legs as 'cancelled'. Used primarily for passenger-initiated cancellations.
Inputs required	• <code>journeyId</code> (string) - Unique identifier for the multi-leg journey • <code>reason</code> (optional string) - Reason for cancellation
Outputs returned	• <code>success</code> (boolean) - True if the cancellation was successful
Interactions with other systems/components	• Database Service - Patches the <code>multiLegJourneys</code> document • Ride Request System Service - Implicitly cancels any pending ride requests related to the journey

9.7 handleTransferTimeout Mutation

Field	Description
What the service does	Manages the state transition when a transfer window expires (though disabled in the current manual flow), setting the overall journey status to 'timeout'.
Inputs required	• <code>journeyId</code> (string) - Unique identifier for the multi-leg journey
Outputs returned	• <code>success</code> (boolean) - True if the status was successfully updated
Interactions with other systems/components	• Database Service - Patches the <code>multiLegJourneys</code> document • Background Task Scheduler - Previously used to trigger this function on timeout

9.8 getJourneyState Query

Field	Description
What the service does	Retrieves the complete current state document for a specific multi-leg journey using its unique ID.
Inputs required	• <code>journeyId</code> (string) - Unique identifier for the multi-leg journey
Outputs returned	• <code>journey</code> (object or null) - The full <code>multiLegJourneys</code> document
Interactions with other systems/components	• Database Service - Queries the <code>multiLegJourneys</code> table

9.9 getActiveJourneyForPassenger Query

Field	Description
What the service does	Identifies and retrieves the single currently active (in_progress) or planned journey for a specific passenger.
Inputs required	• passengerId (Id "taxiTap_users") - The ID of the passenger
Outputs returned	• journey (object or null) - The full active or planned multiLegJourneys document
Interactions with other systems/components	• Database Service - Queries the multiLegJourneys table by passenger ID and status

9.10 cleanupExpiredTransfers Mutation (Internal)

Field	Description
What the service does	An internal function intended to find and mark journeys as 'timeout' if the transfer window expired. Currently disabled as the system transitioned to a manual user-driven flow.
Inputs required	• None
Outputs returned	• cleanedCount (number) - Number of journeys processed (always 0 while disabled) • disabled (boolean) - Indicates the function is currently inactive • reason (string) - Explains why the function is disabled
Interactions with other systems/components	• Database Service - Queries and potentially patches multiLegJourneys table

10 Conclusion

These service contracts provide a comprehensive foundation for the TaxiTap system architecture. Each contract clearly defines the interface, behavior, and integration points for major system services, enabling reliable communication between components and facilitating system maintenance and expansion.